Manual Operations & Runbook (No Agent Required)

This document provides complete, step-by-step instructions to manually start, configure, run, and troubleshoot all components of the **Multi-Stage Agentic SAST Engine** on your local machine.

1. Architecture & Port Reference

The SAST platform consists of three core services and scanning engines:

Service	Port / URL	Technology	Role
Backend API	`http://127.0.0.1:8000`	Python `server.py`	Core scanning engine, pipeline orchestrator, JSON API
Frontend UI	`http://localhost:5173`	React + Vite (`ui/`)	Interactive triaging dashboard, findings explorer
DefectDojo	`http://localhost:8083`	Docker Compose	Vulnerability management & findings import
Joern CPG	CLI / Local	Scala / CPG Engine	Baseline comparison & semantic code analysis

> **Credential Note:** All local environment settings, credentials, and API keys are stored in `.env.local` (which is gitignored).

2. Quick Start: One-Script Method (Recommended)

The repository includes a PowerShell automation script (`services.ps1`) that manages all services automatically.

A. Starting All Services

Open **PowerShell** in the root folder `c:\Users\sunil\Downloads\sast-engine\sast-engine` and execute:

```
# 1. Start backend, dashboard, and DefectDojo containers
.\services.ps1 start
```

B. Checking System Health & Status

```
.\services.ps1 status
```

C. Stopping Services

```
.\services.ps1 stop
```

D. Restarting Services

```
.\services.ps1 restart
```

3. Manual Terminal-by-Terminal Execution

If you prefer running and monitoring each service in its own dedicated terminal window:

Terminal 1: Python Backend API

```
# Navigate to project directory
cd c:\Users\sunil\Downloads\sast-engine\sast-engine

# Activate Python virtual environment
.\.venv\Scripts\Activate.ps1

# Run the backend API server
python server.py
```

Output will indicate: `SAST engine API on http://127.0.0.1:8000`

Terminal 2: Frontend Vite Dashboard

```
# Navigate to the ui directory
cd c:\Users\sunil\Downloads\sast-engine\sast-engine\ui

# Start Vite dev server
npm run dev
```

Open your browser at: http://localhost:5173

Terminal 3: DefectDojo Containers (Docker)

```
# Make sure Docker Desktop is open and running
cd C:\Users\sunil\django-DefectDojo

# Launch containers in background
docker compose up -d
```

DefectDojo Web Portal: http://localhost:8083 (Login: `admin` / `SastEngine2026!`)

4. How to Run Scans Manually

Method A: Through the Web Dashboard (UI)

1. Open http://localhost:5173 in Google Chrome or Microsoft Edge.
2. Under **Run a scan**:
   - Click one of the quick test targets (e.g., `testdata/vuln-flask`, `testdata/vuln-express`, or `testdata/safe-app`), **or** click **Upload Project File / Zip** to scan your own code.
   - Select the engine: `builtin (fast)` or `joern (code property graph)`.
   - Check **compare against Joern baseline** or **push to DefectDojo** if desired.
3. Click the blue **Scan** button.
4. Inspect findings, taint trace graphs, PoC exploits, and automated remediation recommendations in real-time.

Method B: Through the CLI

In PowerShell with virtual environment activated:

```
# Standard scan on test data
python scan.py testdata/vuln-flask

# Scan with full Joern baseline comparison
python scan.py testdata/vuln-flask --engine joern

# Scan and push directly to DefectDojo
python scan.py testdata/vuln-flask --push

# Run without LLM network calls (forces rule-based validator)
python scan.py testdata/vuln-flask --no-llm

# View suppressed false positives and reasons
python scan.py testdata/vuln-flask --show-suppressed
```

5. Configuration File (.env.local) Reference

The file `.env.local` in the root directory controls all engine integration endpoints and keys:

```
# DefectDojo Settings
DEFECTDOJO_URL=http://localhost:8083
DEFECTDOJO_TOKEN=16d7821e440e9434371b043a40dbaa7e2f96fa5f
DEFECTDOJO_USER=admin
DEFECTDOJO_PASSWORD=SastEngine2026!
DEFECTDOJO_DIR=C:\Users\sunil\django-DefectDojo

# Joern CLI Home Directory
JOERN_HOME=C:\Users\sunil\joern-cli

# LLM Validator (OpenAI / Anthropic)
OPENAI_API_KEY=sk-proj-x1JTCTgh1S20lszWnLMACMz310PGL7oHr5F083MjW4_md7UIAmU2B70bJR0BrAG-mvD8zKkBIaT3B
LLM_PROVIDER=openai
OPENAI_MODEL=gpt-4o-mini
```

6. Troubleshooting & Common Issues

Issue 1: Browser shows "127.0.0.1 refused to connect"

- **Cause:** The Python server or Vite dev server is not currently running.
- **Solution:** Run `.\services.ps1 restart` in PowerShell. Make sure to open the UI at `http://localhost:5173` (the web dashboard) rather than opening port 8000 directly.

Issue 2: DefectDojo status says "not connected / connection refused"

- **Cause:** Docker Desktop was closed or containers are in an exited state.
- **Solution:** Launch Docker Desktop, then in PowerShell run:

  ```
  cd C:\Users\sunil\django-DefectDojo
  docker compose up -d
  ```

Issue 3: UI Header shows "validator: offline (openai did not answer)"

- **Cause:** OpenAI API key hit a rate limit (HTTP 429) or has insufficient credit quota.
- **Engine Behavior:** The SAST engine automatically falls back to its deterministic rule engine to ensure scans never fail.
- **Solution:** Add credits to your OpenAI account or update `OPENAI_API_KEY` in `.env.local`, then run a fresh scan.

Issue 4: Port 8000 or 5173 already in use

```
# Force stop any lingering processes on ports 8000 and 5173
.\services.ps1 stop
.\services.ps1 start
```